

The Control Tower Brief

Drill · eight code exhibits · Architecture Atlas · Blueprint Book (40 q) · sealed key · Memory Map

Days 2-4 of the 35-day campaign · Wed 30 Sep - Fri 2 Oct 2026 · 9 study hours

Six objectives. Two of them — **1.3** (pattern selection, where your 50% stop_reason row lands) and **1.5** (decomposition, your 75% row) — get the most pages, the hardest exhibits and eight questions each in the Blueprint Book. The other four are consolidation.

1.1	Business problem → Claude solution	Carried over
1.2	End-to-end architecture + feedback loops	Carried over
1.3	Pattern: workflow / agentic / augmented LLM	SOFT · 50%
1.4	Multi-agent orchestration	Carried over
1.5	Decomposition techniques	SOFT · 75%
1.6	Business value pillars and SLAs	Carried over

THE FIVE LAWS OF DOMAIN 1 · EACH GETS ITS OWN BOX INSIDE, ALL FIVE LIVE ON THE MEMORY MAP

- 1 The constraint picks the pattern, not the ambition.
- 2 Workflows when the path is known; agents when it isn't.
- 3 Every loop needs a budget and an exit.
- 4 Decompose on discovery, not in advance.
- 5 Value is a pillar with a number attached.

HOW TO USE THIS BOOK

Day A reads the Drill and hand-types the exhibits. Day B sits the Blueprint Book cold. Day C reads the Atlas, retakes every miss, and finishes the Memory Map. The key at the back is **sealed**: it is not opened until the Blueprint Book has been sat in one sitting. Every session ends with the miss ledger — the template is the last page.

Holmes & Watson, back on the case. — The game, Watson, is no longer whether you can build the machine. It is whether you would.

DAY A · Wed 30 Sep · 2 h**DRILL DAY**

- 15'** Miss ledger from Mock I — read the D1 rows out loud, family codes first.
- 60'** Chapters 0–4: the brief, the taxonomy, the five workflows, the agent loop. Hand-type E1 and E7; the rest are read tonight, typed across the cycle.
- 30'** Decode E7 line by line. This is the 50% row; it gets the slow treatment.
- 15'** Draft the Memory Map by hand — what would you want in the room?

TONIGHT YOU WILL

Tonight you will: name the seven patterns from memory, and say the seven stop_reason values without looking.

DAY B · Thu 1 Oct · 2 h**BENCH DAY**

- 15'** Miss ledger review. Same ritual.
- 60'** Blueprint Book, all 40 questions, one sitting, key sealed. Underline the tell in every stem before reading options.
- 30'** Sabotage lab on E5 and E8 — break one variable each, predict, run, fix.
- 15'** Ledger every miss: objective, family code, the tell you skipped.

TONIGHT YOU WILL

Tonight you will: know your D1 score by objective, and which family (VG/OE/TR/CM/HL) owns your misses.

DAY C · Fri 2 Oct · 5 h (Gandhi Jayanti)**ATLAS DAY**

- 30'** Miss ledger review — now with Day B rows in it.
- 90'** Architecture Atlas A1–A7. Redraw A4 (the pattern-selector) from memory.
- 90'** Retake every Blueprint Book miss cold. New-territory words → layman glossary.
- 60'** Type the remaining exhibits (E2–E6, E8). Run each DRY_RUN first, then live.
- 30'** Finalise the Memory Map — clean copy, colour grammar. Day 34 reads this page.

TONIGHT YOU WILL

Tonight you will: have a clean D1 ledger and a finished Memory Map. Tomorrow, the Loading Dock (D3).

The Brief — Meera's ask, and why nobody writes a prompt yet

↑ A BRIDGE BEFORE A BLUEPRINT

A city hires an engineer and says: *we need to cross the river*. The amateur starts drawing a suspension bridge, because suspension bridges are magnificent. The engineer first asks: how many cross per day, carrying what, in what weather, and what happens when the bridge is closed? Sometimes the answer is a ferry. Sometimes it is a tunnel. Occasionally it really is the suspension bridge. **Domain 1 is the discipline of asking before drawing** — and the exam fails the candidate who draws the magnificent thing when the ferry met the constraint.

Monday morning, Pune. Meera Nair calls Arjun Bose into the glass room beside the operations floor. Below them, **40 dispatchers** are working **~40,000 shipments a day**. Exceptions — delays, damage, customs holds, refund demands — arrive by email, by portal ticket, and by phone. Meera's ask is one sentence:

"I want an AI layer over this desk. The board approved budget on Friday. Tell me what we build."

Somewhere in the pile is consignment **KF-2481** — Padma Deshmukh's shipment, three days late, about to become the running example of this whole book. Arjun's job this week is not to write a prompt. It is to answer four questions in order, and they are the four questions Domain 1 grades:

#	Arjun must decide	Objective	Chapter
1	What is the actual problem, and would an AI solution beat the current desk on a pillar Meera can defend to the board?	1.1 · 1.6	1, 8
2	Which shape of system: a workflow, an agent, or just an augmented LLM?	1.3	2–4
3	How does the work decompose — fixed steps, or subtasks discovered as tool results come back?	1.5	5
4	If several agents, who orchestrates whom , and where do humans sit in the loop?	1.4 · 1.2	6–7

⚠ TRAP · THE MAGNIFICENT BRIDGE

The exam's favourite D1 wrong answer is the most capable architecture in the option list. A stem that says the steps are the same every time or the desk must answer in two seconds has already picked a simple workflow — choosing the multi-agent system anyway is the **OE** family, and it will appear in at least a quarter of D1 items. But beware the reflex in reverse: when the stem says the subtasks cannot be predicted, the agent IS the right answer, and refusing it is the **TR** family.

Anthropic's own engineering guidance opens with exactly this discipline: find the **simplest solution possible**, and add complexity *only when it demonstrably improves outcomes*. That sentence, in exam form, is Law 1.

LAW 1 · THE CONSTRAINT PICKS THE PATTERN, NOT THE AMBITION

Read the stem for the constraint — the known path, the latency budget, the unpredictable subtasks, the regulated data — and let it choose. An option that is more capable than the constraint demands is not "better"; on this exam it is wrong.

CHAPTER 1 · OBJECTIVE 1.1

From business problem to Claude solution

↑ THE DOCTOR, NOT THE PHARMACIST

Walk into a pharmacy and ask for "the strongest medicine you have" and a good pharmacist refuses — strongest against *what*? A doctor starts from the symptom, the history, the allergy list, and only then reaches for the shelf. Objective 1.1 is being the doctor: the business problem is the patient, and "Claude" is the shelf, not the prescription.

Translating Meera's ask means rewriting it until it names a **workload**, a **constraint**, and a **measure of better**. Watch the sentence sharpen:

Version	The ask	What's still missing
Meera's	"An AI layer over the desk."	Everything — this is ambition, not a problem
Better	"Reduce time-to-resolution on shipment exceptions."	Volume, constraint, target
Exam-grade	"Triage ~1,200 exceptions/day into 6 categories with a draft reply, first touch under 5 min, refunds above ₹10,000 always reaching a human."	Nothing — pattern selection can begin

The exam-grade version already whispers its architecture: *triage into known categories* is a router; *draft reply* is a chained step; *refunds above a threshold to a human* is a gate. A well-translated problem makes objective 1.3 almost mechanical — which is why 1.1 stems reward the option that **clarifies before building**.

WHERE CLAUDE FITS — AND WHERE IT SHOULDN'T

Signal in the stem	Reading	Family if ignored
Judgement over unstructured text at volume (emails, tickets, documents)	Claude-shaped work	—
A deterministic rule already covers it ("if carrier = X, reroute to Y")	Plain code; an LLM adds cost and failure modes	OE
Ambiguity a human currently resolves by asking a colleague	Claude drafts, human confirms — a gate, not a replacement	HL
"Real-time" inside a hard control loop (sortation machinery)	Not an LLM's seat — latency and determinism rule it out	CM

⦿ THE TELL

1.1 stems hide the decision in the verb. "Classify... into six known categories" is not the same problem as "investigate... and decide". The first is a workflow; the second smells of an agent. Underline the verb before you look at a single option.

CHAPTER 2 · OBJECTIVE 1.3A

The augmented LLM — the cell every organism is built from

⬆ ONE CLERK WITH THREE DRAWERS

Picture one clerk at one desk with three drawers: a filing cabinet (**retrieval**), a telephone (**tools**), and a notebook (**memory**). The clerk decides for himself when to open which drawer. He is not a department, not a process chart — one competent clerk, augmented. Every pattern in this book is just this clerk, photocopied and arranged.

Anthropic's taxonomy stacks three floors: the **augmented LLM** (one model call that can retrieve, call tools, and hold memory) → **workflows** (several calls orchestrated through *predefined code paths*) → **agents** (the model *dynamically directs its own process and tool usage*). Hold the italic phrases; they are the exam's dividing lines, quoted almost verbatim in stems.

EXHIBIT E1 · THE AUGMENTED LLM — KAVI TRIAGES ONE EXCEPTION

One call, one tool, no loop beyond a single round-trip. Note the **DRY_RUN** switch: the free probe always precedes the paid call.

e1_augmented_llm.py

```

1  import os, json, anthropic
2
3  DRY_RUN = os.environ.get("DRY_RUN") == "1"      # free probe first, always
4  MODEL = "claude-sonnet-5"                      # current alias, Sep 2026
5
6  tools = [{
7      "name": "get_shipment_status",
8      "description": "Live status for one Kaveri consignment id.",
9      "input_schema": {"type": "object",
10                      "properties": {"consignment_id": {"type": "string"}},
11                      "required": ["consignment_id"]},
12  }]
13
14  def fake_status(cid):                          # canned world for DRY_RUN
15      return {"id": cid, "status": "CUSTOMS_HOLD", "days_late": 3}
16
17  def triage(email_text):
18      if DRY_RUN:
19          return {"category": "customs", "urgency": "high"}
20      client = anthropic.Anthropic()
21      msg = client.messages.create(
22          model=MODEL, max_tokens=500,
23          system="You triage Kaveri shipment exceptions. Reply as JSON "
24                "{category, urgency}. Categories: delay, damage, customs, "
25                "refund, address, other.",
26          tools=tools,
27          messages=[{"role": "user", "content": email_text}])
28      if msg.stop_reason == "tool_use":          # the clerk opened a drawer
29          call = next(b for b in msg.content if b.type == "tool_use")
30          result = fake_status(call.input["consignment_id"])
31          msg = client.messages.create(
32              model=MODEL, max_tokens=500, tools=tools,
33              messages=[{"role": "user", "content": email_text},
34                        {"role": "assistant", "content": msg.content},
35                        {"role": "user", "content": [{"type": "tool_result",
36                                                       "tool_use_id": call.id,
37                                                       "content": json.dumps(result)}]})
38      return msg.content[0].text
39
40  print(triage("Where is KF-2481? Three days late! - Padma"))

```

 PREDICT BEFORE RUN

Before running with DRY_RUN=1: what prints, and how many paid API calls were made? Then before the live run: which branch fires — does the model call the tool for Padma's email, and why would it?

Prediction, in the notebook, before running:

.....

EXPECTED OUTPUT

```

$env:DRY_RUN="1"; python e1_augmented_llm.py
{'category': 'customs', 'urgency': 'high'}
# live run (DRY_RUN unset):
{'category': 'customs', 'urgency': 'high'} # after one tool round-trip

```

DECODE · LINE BY LINE, ONE STORY

1–4	One story runs through this file: Padma's angry email arrives at the desk . Line 3 is the light switch by the door — flip DRY_RUN on and the whole building runs on battery: no bill, no network. Line 4 names the clerk: a current model alias, so the workbook doesn't rot when versions move.
6–13	The telephone on the desk. A tool is a promise, not code: name, description, and a JSON schema saying “to call me, give me a consignment_id string”. The model reads this promise like a phone book entry. The required list is the phone book saying “no id, no call” — delete it in the sabotage lab and watch what the model does.
15–16	The canned world. When the building is on battery, someone still has to answer the phone — fake_status is the answering machine, always replying “customs hold, three days late” so your prediction has something solid to be checked against.
18–20	The triage function opens. First move: if we're on battery, return the canned verdict and stop. The paid clerk is never disturbed until the free rehearsal has run.
21–28	The one paid call. system is the clerk's standing orders — reply as JSON, six categories only. tools hands over the phone book. The email itself rides in messages as the user turn.

29	The hinge of the whole chapter. The response's <code>stop_reason</code> is the clerk's note on top of the file. " tool_use " means "I picked up the phone — connect me." Anything else means the file is finished. Chapter 4 spends six pages on this one field.
30–32	Reading the phone request: find the <code>tool_use</code> block, dial the (canned) number, get the status back. In production this line is the real MCP call to <code>kaveri_mcp.py</code> .
33–38	The second call is the whole conversation so far, replayed plus one new user turn carrying the <code>tool_result</code> — matched to the request by <code>tool_use_id</code> , the way a courier receipt matches a parcel. The API is stateless: forget to replay the assistant turn and the clerk has amnesia. That's sabotage #1.
39–41	The clerk's final verdict comes back as plain text (our JSON), and Padma's email has been triaged: one exception, ≤2 paid calls, no loop, no committee. This is the cell. Everything after this chapter is arrangement.

✂ SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 12: delete `"required": ["consignment_id"]`. Predict: does the code crash, or does the model sometimes call the tool with empty input and `fake_status` throw a `KeyError`? Run live twice. Lesson: **the schema is a guardrail you own; the model's compliance is a behaviour you rent**. Fix it back. One variable, one lesson.

⚡ DRIFT · PREFILLED ASSISTANT TURNS

Workbook / common teaching: Older material forces JSON by pre-filling the assistant's reply with `"{"`.

Current docs (checked Sep 2026): Assistant-message prefill is unsupported on newer models (returns 400). The current route is structured outputs via `output_config.format` with a JSON schema — or, as here, instructions plus validation in code.

CHAPTER 3 · OBJECTIVE 1.3B

The five workflows — arrangements of clerks, paths fixed in code

A **workflow** is several augmented-LLM calls whose sequence is decided by *your code*, before any model speaks. The model fills in steps; it never chooses them. Five named arrangements cover the exam. Learn each as: **shape** → **when** → **Kaveri scene** → **trap**.

3.1 PROMPT CHAINING · E2

↑ THE ASSEMBLY LINE

Three clerks in a row: the first sorts the letter, the second drafts the reply, and between them stands a supervisor with a checklist who bins anything malformed. Nobody improvises the order. Accuracy up, latency up — you queued three desks.

When: the task decomposes into fixed subtasks known in advance, and you'll trade latency for accuracy. The programmatic gate between steps is the pattern's signature.

```
e2_chaining.py (ask() = one messages.create; DRY_RUN inside)

1  VALID = {"delay", "damage", "customs", "refund", "address", "other"}
2
3  def classify(email):                      # step 1 - small, cheap, focused
4      out = ask(MODEL_SMALL, "One word, the category only: " + email)
5      return out.strip().lower()
6
7  def gate(category):                     # the checklist between desks
8      if category not in VALID:           # malformed -> stop the line
9          raise ValueError(f"bad category: {category}")
10     return category
11
12  def draft_reply(email, category):        # step 2 - sees step 1's output
13     return ask(MODEL_MAIN,
14         f"A {category} exception. Draft a courteous reply, "
15         f"under 120 words, no promises about compensation:\n" + email)
16
17  reply = draft_reply(EMAIL, gate(classify(EMAIL)))
18  print(reply)
```

DECODE · LINE BY LINE, ONE STORY

- 1 The story continues: **Padma's email now rides an assembly line**. `VALID` is the supervisor's laminated checklist — six words, nothing else passes.

3–5	Desk one only sorts. Note the model choice: a small, cheap model is enough to say “customs”. The chain lets each desk be exactly as smart as its step needs — that's the cost story D4 will grade you on later.
7–10	The gate is code, not model . If desk one mumbles “customs-ish”, the line stops with a ValueError instead of desk two writing a confident reply to a nonsense category. Deterministic checks between steps are the whole reason chaining beats one big prompt.
12–15	Desk two writes to Padma, told the category and told the boundaries (length, no compensation promises — a guardrail D5 will formalise). It never re-decides the category; the line has one direction.
17–18	The line assembled in one expression: classify → gate → draft. Your code owns this order forever. When the stem says <u>the steps are the same every time</u> — this, not an agent, is the answer.

⚠ TRAP · CHAIN WHERE A LOOKUP WOULD DO

If the category is already a field on the ticket, desk one is an LLM re-deriving what a database column knows. OE wears a lab coat here: it looks diligent. Check the stem for data that already exists before you spend a model call creating it.

3.2 ROUTING · E3

↑ THE HOSPITAL TRIAGE NURSE

One nurse at the door, thirty seconds per patient, sending each to the right specialist ward. The nurse doesn't treat anyone. Cheap decision at the front; expensive expertise only where it's needed.

When: distinct input categories exist, each better served by a different prompt or model, and classification is reliable. Routing separates concerns: improving the refund specialist can't break the damage specialist.

e3_routing.py

```

1  SPECIALISTS = {
2      "refund": ("claude-opus-5-5", "You resolve refund disputes. Cap: "
3          "acknowledge, verify, never promise."),
4      "customs": ("claude-sonnet-5", "You explain customs holds and next "
5          "steps for Indian import consignments."),
6      "delay": ("claude-haiku-4-5-20251001", "You write short delay "
7          "apologies with the new ETA."),
8  }
9
10 def route(email):
11     cat = gate(classify(email))          # reuse E2's nurse + checklist
12     model, system = SPECIALISTS.get(cat, SPECIALISTS["delay"])
13     return ask(model, system + "\n\n" + email)
14
15 print(route(EMAIL))

```

DECODE · LINE BY LINE, ONE STORY

1–8	Three wards, three door signs. Read the model column with D2 eyes: the refund ward gets the big model (money, judgement), delays get the small one (an apology has a template's heart). Routing is where model-selection stops being one decision and becomes a decision per category.
10–13	The nurse is E2's classifier, reused, gate and all. One new idea only: <code>.get(cat, default)</code> — an unmapped category falls to a safe ward instead of a KeyError at 2 a.m. Silent failure's cheap insurance.
15	One email in, one specialist out. The path was chosen by code reading one word — the model never saw a menu of wards. That's what keeps this a workflow, not an agent.

⚠ TRAP · ROUTER GRADING ITS OWN HOMEWORK

A stem offering “let the specialist model also verify the routing was correct” is inviting a circle: the expensive call re-doing the cheap call's job. If routing accuracy is the worry, the answer lives in D4 (an eval set for the classifier), not in doubling the pipeline.

3.3 PARALLELIZATION · E4 — SECTIONING AND VOTING

↑ THREE SCOUTS, OR THREE JUDGES

Send three scouts down three different roads at once and the answer arrives when the slowest scout returns — that's **sectioning**. Ask three judges the same question and take the majority — that's **voting**. Same trick (simultaneity), opposite purposes: coverage versus confidence.

e4_parallel.py

```

1  import asyncio
2
3  async def check(carrier, cid):          # one scout, one road
4      return await ask_async(MODEL_SMALL,
5                             f"ETA for {cid} via {carrier}? One line.")
6
7  async def sectioning(cid):              # coverage: different subtasks
8      return await asyncio.gather(check("BlueDart", cid),
9                                  check("Delhivery", cid),
10                                 check("IndiaPost", cid))
11
12  async def voting(claim):                # confidence: same subtask x3
13      verdicts = await asyncio.gather(*[
14          ask_async(MODEL_MAIN,
15                  "APPROVE or REJECT only. Refund claim:\n" + claim)
16          for _ in range(3)])
17      return max(set(verdicts), key=verdicts.count)  # majority wins
18
19  print(asyncio.run(sectioning("KF-2481")))
20  print(asyncio.run(voting(CLAIM_TEXT)))

```

DECODE · LINE BY LINE, ONE STORY

1–5	The story forks: KF-2481's rescue needs three carrier quotes . Each check() is one scout with one road. async means a scout who doesn't make the others wait at the gate while he walks.
7–10	asyncio.gather releases all three at once. Wall-clock time = the slowest scout, not the sum — with three 2-second calls, ~2 s instead of ~6 s . When a stem waves a latency budget at independent subtasks, this is the tell.
12–17	Voting: same claim, three independent verdicts, majority rules. Cost triples — 3× tokens for one decision — so voting must be spent where being wrong is expensive, like refunds, not where it's cheap, like apologies.
19–20	Both halves run. Notice what did NOT happen: no model chose to parallelize. Your code drew the fork. Workflow.

⚠ TRAP · VOTING ON FACTS

Voting raises confidence on judgement; it cannot vote a fact into existence. Three models guessing an ETA agree on a hallucination as happily as on the truth — the scout for facts is a tool call, not a ballot. TR family: right mechanism, wrong axis.

3.4 ORCHESTRATOR-WORKERS · E5 (THIS IS 1.5'S ENGINE — FULL DECODE IN CHAPTER 5)

👤 THE SITE FOREMAN

A foreman walks the site each morning and *writes that day's task list himself* — nobody handed him one. Masons, electricians, plumbers each get a chit; the foreman reads their reports and synthesises. The task list is different every morning because the site is.

When: subtasks can't be predicted in advance — the model must decompose. This is the boundary stone between workflows and agents: the *structure* (delegate → work → synthesise) is still fixed code, but the *content* of the plan is the model's. E5's code and its line-by-line decode live in chapter 5, where objective 1.5 needs them.

3.5 EVALUATOR-OPTIMIZER · E6

👤 THE WRITER AND THE EDITOR

One clerk drafts; a second, colder clerk marks it against a rubric and hands it back. Two rounds of this and the letter is noticeably better. But without a page limit the pair would polish forever — so the editor's desk has a rule: two revisions, then it ships.

e6_evaluator_optimizer.py

```

1  def draft_and_polish(email, max_rounds=2):
2      draft = ask(MODEL_MAIN, "Reply to this exception email:\n" + email)
3      for round_no in range(max_rounds):          # the budget
4          verdict = ask(MODEL_MAIN,
5              "Rubric: tone courteous? facts only? under 120 words? "
6              "Reply PASS, or one line of specific criticism:\n" + draft)
7          if verdict.strip() == "PASS":          # the exit
8              break
9          draft = ask(MODEL_MAIN,
10             f"Revise. Criticism: {verdict}\nDraft:\n{draft}")
11     return draft

```

DECODE · LINE BY LINE, ONE STORY

1–2	The letter to Padma gets an editor. First draft is free-form; everything after answers to the rubric.
3	The budget: at most 2 rounds. Law 3 in four characters. An unbounded while-loop here is the classic silent way to spend ₹400 polishing a delay apology.
4–7	The editor sees the rubric and the draft , nothing else. Clear evaluation criteria are the pattern's entry fee — no rubric, no evaluator, just two models exchanging vibes.
7–8	The exit: PASS breaks early. Budget AND exit — E7's agent loop will need exactly the same two organs, grown larger.
9–11	The revision call carries the criticism and the draft — targeted feedback on a specific failure, which is precisely the iterative-refinement discipline your 50% CCAR-F row was testing. The letter that ships is the last draft, PASS or budget-out.

⚠ TRAP · EVALUATOR WITHOUT CRITERIA

“Have a second model review it” with no rubric is the SD ghost of CCAR-F wearing a new coat — it sounds diligent and measures nothing. If the stem names no criteria, the evaluator option is decoration, and the correct answer usually fixes the first prompt instead.

LAW 2 · WORKFLOWS WHEN THE PATH IS KNOWN; AGENTS WHEN IT ISN'T

All five arrangements share one property: code owns the path. Chaining and routing for known categories; parallel for independent subtasks or votes; orchestrator-workers when the plan itself must be model-made; evaluator-optimizer when a rubric exists. The moment no path can be fixed in advance — the model must look at each result to choose its next act — you have left this chapter.

CHAPTER 4 · OBJECTIVE 1.3C

The agent — and the loop that is your 50% row

↑ THE DETECTIVE ON A RETAINER

A workflow clerk follows a checklist. A detective is hired differently: “find out why KF-2481 is stuck — here are your tools, here is your expense cap, report when done.” Each clue decides the next move; nobody scripted the investigation. The retainer has two clauses every stem cares about: the **expense cap** (budget) and **report when done** (exit). A detective without both is not employed — he is loose.

An **agent** is a model dynamically directing its own process and tool usage. Mechanically it is one thing only: a **loop around messages.create that keeps going while the model asks for tools and stops when it doesn't**. The field that steers the loop is **stop_reason** — and your Foundations report says this exact mechanism cost you half its marks. So: the slowest, fullest treatment in the book.

EXHIBIT E7 · THE AGENT LOOP, EVERY BRANCH EXPLICIT

```

e7_agent_loop.py (TOOLS = the two kaveri_mcp schemas)

1  import os, json, anthropic
2  MODEL = "claude-sonnet-5"; BUDGET = 6          # the retainer's cap
3  DRY_RUN = os.environ.get("DRY_RUN") == "1"
4
5  def run_tool(call):                             # detective's toolbag
6      if call.name == "get_shipment_status":
7          return {"status": "CUSTOMS_HOLD", "missing": "e-way bill"}
8      if call.name == "find_alternate_carriers":
9          return ["BlueDart", "Delhivery"]
10     return {"error": f"unknown tool {call.name}"}
11
12 def investigate(brief):
13     msgs = [{"role": "user", "content": brief}]
14     client = None if DRY_RUN else anthropic.Anthropic()
15     for turn in range(1, BUDGET + 1):             # bounded, always
16         if DRY_RUN:
17             print(f"[dry] turn {turn}"); return "dry-run verdict"
18         r = client.messages.create(model=MODEL, max_tokens=1000,
19                                   tools=TOOLS, messages=msgs)
20         match r.stop_reason:
21             case "tool_use":                     # the loop's fuel
22                 msgs.append({"role": "assistant", "content": r.content})
23                 results = [{"type": "tool_result", "tool_use_id": b.id,
24                             "content": json.dumps(run_tool(b))}]
25                 for b in r.content if b.type == "tool_use"
26                 msgs.append({"role": "user", "content": results})
27             case "end_turn":                     # the natural exit
28                 return r.content[0].text
29             case "max_tokens":                   # truncated, not done
30                 msgs.append({"role": "assistant", "content": r.content})
31                 msgs.append({"role": "user", "content": "Continue."})
32             case "pause_turn":                  # server loop paused
33                 msgs.append({"role": "assistant", "content": r.content})
34             case "refusal":                     # declined - do not retry blindly
35                 return escalate(brief, "model refused")
36             case _:                             # stop_sequence, context exceeded
37                 return escalate(brief, f"unhandled: {r.stop_reason}")
38     return escalate(brief, "budget exhausted")    # cap reached: human takes it
39
40 def escalate(brief, why):
41     return f"ESCALATED to Meera's desk ({why}): {brief[:60]}"
42
43 print(investigate("Why is KF-2481 stuck, and what are our options?"))

```

PREDICT BEFORE RUN

DRY_RUN=1 first: how many lines print, and why exactly one? Then live: KF-2481 is in a customs hold — write the sequence of stop_reason values you expect across the turns (e.g. tool_use → tool_use → end_turn), and which branch you believe never fires today.

Prediction, in the notebook, before running:

.....

EXPECTED OUTPUT

```

$env:DRY_RUN="1"; python e7_agent_loop.py
[dry] turn 1
dry-run verdict
# live, typical trace:
# turn 1 stop_reason=tool_use (get_shipment_status)
# turn 2 stop_reason=tool_use (find_alternate_carriers)
# turn 3 stop_reason=end_turn
KF-2481 is held at customs for a missing e-way bill. Options: file the
bill (fastest), or re-route via BlueDart/Delhivery once released...

```

DECODE · LINE BY LINE, ONE STORY

- | | |
|-------------|--|
| 1–3 | The retainer is signed: the detective takes the KF-2481 case . BUDGET = 6 is the expense cap in line two of the contract — not an afterthought at the bottom. Law 3 is written before the loop exists. |
| 5–10 | The toolbag. Same promise-shape as E1's phone book, now two tools. Note line 10: an unknown tool name returns a structured error result, not an exception — the detective learns his tool failed and can adapt. Crashing the loop would teach him nothing. |

12–14	The case file opens with one user turn: the brief. Everything the investigation learns will be appended here, because the API is stateless — the file IS the memory.
15	for turn in range(1, BUDGET+1) — the loop is bounded by construction. The exam's favourite wrong option runs while True with the budget checked “somewhere else”. Somewhere else is nowhere.
18–19	One paid call per turn: the entire case file goes up, one decision comes back. The response's stop_reason is the detective's note clipped to the front — and the match below reads that note. This match statement is the whole objective.
21–26	tool_use — “I need the phone.” Three moves, in order: file his request (assistant turn), run every tool block he asked for, file the receipts (tool_result matched by tool_use_id). Miss the first move and the API rejects the receipts as orphans. The loop then comes round again — the model's own reply decides whether there is a next turn . That sentence is your 50% row, verbatim.
27–28	end_turn — “case closed, here's my report.” The natural exit. Return the text; do not loop again out of politeness. A loop that calls the API once more after end_turn is paying for a conversation with someone who already left.
29–31	max_tokens — the report hit the page limit mid-sentence. Truncated is not finished: file what arrived and say “Continue.” Treating max_tokens as success ships half a verdict — the SF ghost's favourite door.
32–33	pause_turn — a server-side tool loop pausing for breath; return the content as-is to resume. Newer value; most workbooks don't know it exists.
34–35	refusal — the detective declines the case. The wrong reflex is auto-retry; the right one is a different path (here: a human).
36–37	The wildcard keeps the match total: stop_sequence, model_context_window_exceeded, and whatever ships next quarter all land in escalation instead of an unhandled crash. A total match is what “orchestration-layer safeguard” looks like at line level — the thing your 100% row was about, meeting the thing your 50% row was.
38	The loop's own floor: budget spent, case not closed → a completed resolution or a human hand-off, never a silent stop . Every session ends in one of the two. That is the sentence CCAR-F graded at 100% and CCAR-P will grade again.
40–43	Escalation is a first-class outcome with a reason attached — not an error string. Meera's desk learns why the machine gave up, which is what makes the hand-off a Domain 6 handoff package in miniature.

⇒ DRIFT · HOW MANY STOP_REASON VALUES EXIST?

Workbook / common teaching: Most study material teaches four: end_turn, max_tokens, stop_sequence, tool_use.

Current docs (checked Sep 2026): Seven are documented: those four plus **pause_turn** (server-tool loop paused, default limit 10 iterations — send content back to continue), **refusal** (declined; read stop_details), and **model_context_window_exceeded** (treat as truncated). All arrive as HTTP 200 — a stop_reason is a status, never an error.

⚠ TRAP · THE LOOP THAT NEVER ASKS WHY IT STOPPED

Distractors love code that checks `if r.stop_reason == "tool_use"` and treats *everything else* as the finished answer. max_tokens then ships truncations; refusal ships an apology as a verdict. The tell in such stems is an else that swallows. Family: SF grown up — at Professional it's called a diagnosis question (D4.4) too.

LAW 3 · EVERY LOOP NEEDS A BUDGET AND AN EXIT

E6 had max_rounds and PASS. E7 has BUDGET and end_turn — plus escalation as the floor beneath both. Two organs, every loop, no exceptions: when a stem's code lacks one, that absence IS the answer.

SO: WORKFLOW, AGENT, OR AUGMENTED LLM? — THE 1.3 DECISION IN ONE TABLE

The stem says	Choose	Because
Steps identical every time; accuracy over latency	Prompt chaining	Fixed path + gates
Known categories, different handling each	Routing	Cheap sort, specialist wards
Independent subtasks + a latency budget	Parallel (sectioning)	Slowest-scout wall clock
High-stakes judgement, tolerance for 3× cost	Parallel (voting)	Confidence by majority
Subtasks unpredictable, but delegate→synthesise fixed	Orchestrator-workers	Model plans, code steers
Clear rubric, drafts improve measurably	Evaluator-optimizer	Criteria exist
Path itself unknowable; each result steers the next act	Agent	Model directs, budget + exit
One question, tools at hand, no multi-step anything	Augmented LLM	The cell suffices

CHAPTER 5 · OBJECTIVE 1.5

Decomposition — plans that grow when the world talks back

↑ PACKING FOR A TREK VS. RUNNING THE TREK

You can pack for a Himalayan trek in advance — the list is knowable. You cannot *walk* it in advance: at the river crossing you learn the bridge is out, and the plan grows a detour. **Static decomposition** is packing: fixed subtasks, E2's assembly line. **Dynamic decomposition** is walking: subtasks appear because a result revealed them. Your 75% row was exactly this distinction — a plan that adapts versus a sequence that executes regardless.

EXHIBIT E5 · ORCHESTRATOR-WORKERS — THE FOREMAN WRITES THE PLAN

e5_orchestrator_workers.py

```

1  def orchestrate(brief):
2      plan = ask_json(MODEL_MAIN,          # the model WRITES the plan
3          "Brief:\n" + brief + "\n"
4          "List 2-5 subtasks as JSON: "
5          '["id": 1, "task": "...", "needs": []], ...')
6      findings = {}
7      for sub in plan:                    # code STEERS the plan
8          deps = {i: findings[i] for i in sub["needs"]}
9          findings[sub["id"]] = ask(MODEL_SMALL,
10             f"Subtask: {sub['task']}\nKnown: {json.dumps(deps)}")
11      return ask(MODEL_MAIN,             # synthesis sees everything
12          "Brief:\n" + brief + "\nFindings:\n" + json.dumps(findings)
13          + "\nSynthesise one recommendation for Meera.")

```

DECODE · LINE BY LINE, ONE STORY

- | | |
|-------|---|
| 1–5 | Meera's brief — “KF-2481's customer is threatening to leave; fix it” — goes to the foreman, whose first paid act is writing the task list . Compare E2, where VALID and the function order were typed by Arjun: here the subtasks are model-made, different for every brief. This single line is the boundary between objective 1.3's workflows and 1.5's decomposition. |
| 6–10 | The site book. Each subtask runs with its needs — the findings it depends on — injected. A worker checking “can we re-route?” reads the status finding first. Workers are small models: the foreman thinks, the crew executes, the payroll stays sane. |
| 11–13 | Synthesis: the foreman reads every chit and writes ONE recommendation. Not a list of findings — a decision. (Independent subtasks here could be E4's gather; patterns compose.) |

EXHIBIT E8 · RE-PLANNING — THE BRIDGE IS OUT

E5's plan, once written, still executes fixedly. E8 adds the trek's truth: a finding can **change the plan**. This is dynamic decomposition complete — and the exhibit to type slowest.

e8_replanning.py

```

1  def replan(brief, max_new=3):          # cap on growth, too
2      tasks = plan_from(brief)           # E5's opening move
3      findings, grew = {}, 0
4      while tasks:
5          sub = tasks.pop(0)
6          result = run_worker(sub, findings) # E5's worker call
7          findings[sub["id"]] = result
8          verdict = ask_json(MODEL_MAIN,   # the walk-and-look
9              "Given this new finding, does the remaining plan still "
10             "hold? Reply {"ok": true} or {"ok": false}, "
11             "\"add\": [new subtasks], \"drop\": [ids]}\n"
12             + json.dumps({"finding": result, "remaining": tasks}))
13         if not verdict["ok"] and grew < max_new:
14             tasks = [t for t in tasks
15                 if t["id"] not in verdict.get("drop", [])]
16             tasks += verdict.get("add", []) # the detour appears
17             grew += len(verdict.get("add", []))
18     return synthesise(brief, findings)   # E5's closing move

```

PREDICT BEFORE RUN

Seed the canned world so subtask 2's result reveals a customs hold that subtask 3 (a re-route quote) doesn't account for. Predict: which subtasks get dropped, what gets added, and the final value of grew. Then run and check the trace against your notebook.

Prediction, in the notebook, before running:

EXPECTED OUTPUT

```
# trace, canned world:
# run 1: status check    -> CUSTOMS_HOLD, missing e-way bill
# replan: ok=false drop=[3] add=["file e-way bill", "notify Padma of ETA"]
# run 2: file e-way bill -> filed, release ~24h
# run 3: notify Padma    -> draft ready
# grew=2, budget max_new=3 not exhausted
RECOMMENDATION: file the e-way bill today; hold the re-route (costlier
than the 24h release); send Padma the revised ETA with a goodwill note.
```

DECODE · LINE BY LINE, ONE STORY

1–3	The trek begins with E5's packed list — and a second budget : <code>max_new = 3</code> . A plan that can grow needs a cap on growth, or re-planning becomes the unbounded loop wearing a planner's hat. Law 3 applies to plans, not just turns.
4–7	Walk one step: take the front task, run it with the findings so far, file the result. So far, E5 with a queue.
8–12	The walk-and-look. After every finding, one focused question: does the rest of the plan survive this? The model sees the new finding and the remaining tasks — not the whole history. Cheap, specific, structured. This call is what “subtasks that adapt as new information is discovered” means in running code.
13–17	The bridge is out: customs hold discovered → the re-route quote (pointless until release) is dropped, “file e-way bill” and “notify Padma” appear. <code>grew</code> counts the detours against the cap. The plan changed because a result said so — your 75% row, closed.
18	Same synthesis as E5. Decomposition patterns end the same way they must: one recommendation, traceable to findings.

SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 13: change `grew < max_new` to `True`. Seed the canned replanner to always return `ok=false` with one new subtask. Predict what happens; run with a print in the loop; count the turns before you kill it. Lesson: **the growth cap is the difference between a planner and a spiral**. Fix it.

LAW 4 · DECOMPOSE ON DISCOVERY, NOT IN ADVANCE

When subtasks are knowable, decompose in code (chaining) — cheapest and most testable. When they aren't, let the model write the plan (orchestrator-workers). When results can invalidate the plan, re-plan on findings — with a growth cap. The exam grades WHICH of the three the stem's uncertainty actually requires. Escalating uncertainty, escalating machinery; never more than the stem demands.

⚠ TRAP · DECOMPOSING THE KNOWN

A stem whose subtasks are listed in the stem itself does not need orchestrator-workers — the decomposition already exists; paying a model to rediscover it is OE with extra steps. Dynamic machinery is for uncertainty that survives reading the brief twice.

CHAPTER 6 · OBJECTIVE 1.4

Multi-agent systems — when one detective isn't enough**↑ THE AGENCY, NOT THE LONE DETECTIVE**

Some cases outgrow one detective: the customs specialist, the carrier negotiator and the customer handler each need different tools, different briefs, different clearances. An agency has one partner who assigns cases and reads reports — and a strict rule: **field notes stay in the field office; only the report crosses the partner's desk**. That rule is context isolation, and it is why agencies scale where lone detectives drown.

Multi-agent = agents delegating to agents. In the Agent SDK, subagents are separate agent instances the orchestrator spawns; each runs in its own conversation. Four properties carry the exam weight:

Property	Mechanism	Why the exam cares
Context isolation	Subagent explores in its own window; only its final message returns to the parent	A research subagent can read 40 documents without flooding the orchestrator — D2's token economics, solved architecturally
Parallelism	Independent subagents run concurrently	Three reviews finish in the slowest one's time — E4's logic at agent scale
Specialised prompts	Each subagent gets its own system prompt and expertise	The customs agent's SOP knowledge would be noise in the negotiator's context
Tool restriction	Each subagent gets only its own tools; an omitted tool simply doesn't exist for it	The customer-facing agent physically cannot call process_refund — config beats instruction, again

The delegation brief is everything: a subagent starts **fresh** — no parent history — so the orchestrator's prompt to it must carry every path, id and decision it needs. Structured, complete handoffs between agent steps: your other 100% CCAR-F row, now load-bearing at Professional.

CHOOSING A TOPOLOGY

Topology	Shape	Reach for it when	It hurts when
Single agent	One loop, all tools	Tools ≤ a handful, one expertise	Context bloats; one prompt serves three masters
Orchestrator + subagents	Partner assigns, field offices work	Distinct expertises; parallelisable; isolation valuable	Subtasks so entangled every report needs every other's field notes
Peer handoff	Agent A completes, passes baton to B	Sequential phases with a clean break (triage shift → resolution shift)	The baton is informal — an under-specified handoff loses authorization state

⇄ DRIFT · GUARDRAILS AROUND SUBAGENT GROWTH

Workbook / common teaching: Common teaching: “subagents can nest and parallelise” with no numbers attached. **Current docs (checked Sep 2026):** Current SDK documents hard caps you set: spawn depth (default **3** layers, 1 = no nesting), concurrency (default **20** at once), and a per-query spend cap (`max_budget_usd`) that refuses new spawns, stops background subagents and ends the query at the limit. Budgets-and-exits, at fleet scale. Also renamed: the spawning tool now appears as **Agent** in `tool_use` blocks (older versions said Task).

⚠ TRAP · A COORDINATOR OVER THE COORDINATOR

Carrying S3's inversion forward: adding a subagent is often correct — adding a second orchestrator to manage the orchestrator almost never is. When two options both delegate, prefer the one with fewer layers and the same isolation. And remember the six places in this book where the agent/subagent genuinely IS the answer: unpredictable subtasks, isolation, parallel expertise.

CHAPTER 7 · OBJECTIVE 1.2

End-to-end — input, processing, output, and the loop nobody draws

⬆ THE RESTAURANT, NOT THE RECIPE

A recipe is processing. A restaurant is end-to-end: the door (who gets in, in what state), the kitchen, the pass where the head chef tastes before plates leave, and — the part amateurs forget — the comment cards that change next month's menu. An architecture diagram without the comment cards is a recipe pretending to be a restaurant.

Objective 1.2 asks for the full arc. For the Kaveri control tower (drawn full-page as Atlas A3):

Stage	Kaveri concretely	The decision that lives here
Input	Email, portal, phone-transcript exceptions → normalised into one ticket schema	Validation and PII scrubbing happen HERE, before any model — not mid-pipeline
Processing	Router (E3) → specialist chains (E2) → agent for the tangled cases (E7)	Patterns compose; each tier as simple as its slice allows
Output	Draft replies, re-route orders, refund recommendations	The human gate: refunds ≥ ₹10,000 pause for Meera's desk (D5 will formalise)
Feedback	Dispatcher edits diffed vs drafts; outcomes logged; eval set grows weekly (D4)	Without this loop the system is frozen at launch quality forever

⦿ THE TELL

1.2 stems love an architecture that is complete except for one absent arc — usually feedback or the human gate. The right option is rarely “more processing”; it is the missing arc. Count the four stages against every option before judging any of them clever.

CHAPTER 8 · OBJECTIVE 1.6

Value pillars — the boardroom's five words

⬆ THE HOUSEHOLD BUDGET ARGUMENT

“We should get a dishwasher” convinces nobody. “We hand-wash 90 minutes a day; a ₹40,000 machine returns that time inside five months” ends the argument. Same purchase, but the second sentence names a pillar (time), a number (90 minutes), and a horizon (five months). Meera's board hears dishwasher-arguments all day; only numbered pillars survive.

The exam's five pillars, applied to the control tower. A solution is *aligned* when its design choices trace to a pillar with a number:

Pillar	Kaveri number	The design choice it justifies
Efficiency	First touch 4 h → under 5 min	Routing + chaining tier for the routine 80%
Cost	Model spend $\leq$ ₹6/exception avg	Small models at the door (E3); big ones only where money moves
Productivity	Dispatchers 3× throughput, editing not drafting	Draft-plus-gate output design, not auto-send
Performance / SLA	99% of drafts within 30 s	Parallel carrier checks; agent tier reserved for the tangled few
Transformation	Customs-hold pattern reports → a new pre-clearance service line	The feedback log is a product, not exhaust

⚠ TRAP · THE PILLAR SWAP

TR's home ground. The stem fixes one pillar — a 2-second SLA, a cost ceiling — and one option optimises a different, worthier-sounding pillar (“maximise accuracy with a 3-model voting ensemble”). Wrong. The stated pillar is the contract; the exam pays no marks for nobler physics than the customer ordered.

LAW 5 · VALUE IS A PILLAR WITH A NUMBER ATTACHED

“Improves customer experience” is a wish. “Cuts first-touch from 4 hours to 5 minutes at $\leq$ ₹6 per exception” is an architecture requirement — and every D1 option can be tested against it. In 1.6 stems, the option that names measure + number + horizon beats the one that names technology.

CHAPTER CLOSE — WHAT DAY B WILL TEST

Forty questions tomorrow. Six per objective, eight each for 1.3 and 1.5. Underline the tell before the options; name the family of every distractor you reject. The key stays sealed until question 40. — H.

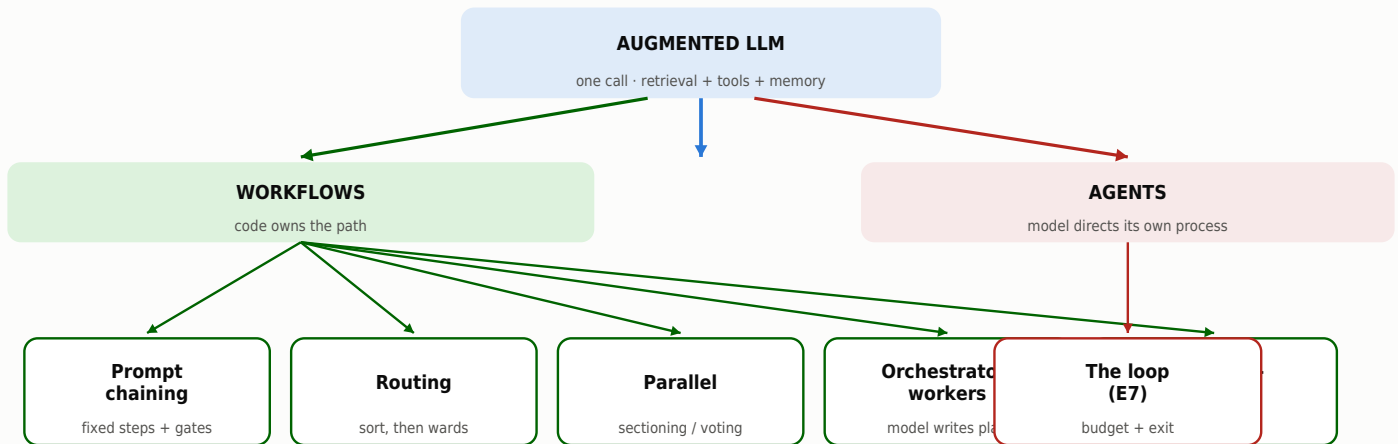
PART TWO

The Architecture Atlas

A1-A7 · the pictures the stems describe in words · Day C redraws A4 from memory

The taxonomy tree — three floors, seven names

Everything in Domain 1 hangs off this tree. If a stem's answer isn't one of these seven leaves, re-read the stem.



© THE TELL

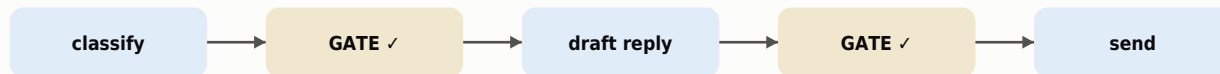
Quoted dividing lines, near-verbatim in stems: workflows = “orchestrated through predefined code paths”; agents = “dynamically direct their own processes and tool usage”. Match the stem's phrase to the floor, then pick the leaf.

ATLAS A2

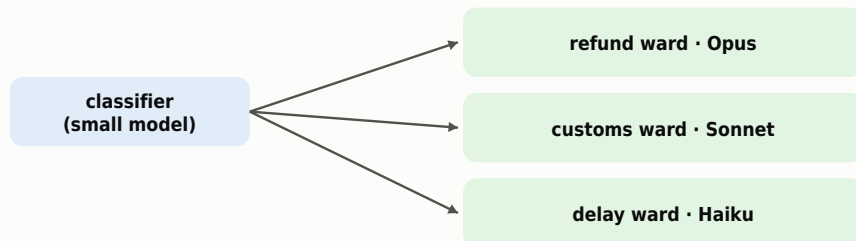
The five workflows — one shape each, Kaveri-labelled

Learn the shapes, not the prose: on the exam you will recognise a diagram described in words.

1 · PROMPT CHAINING — accuracy over latency



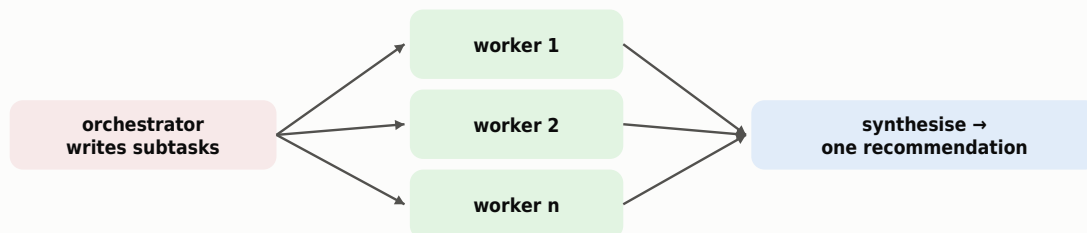
2 · ROUTING — cheap sort, specialist wards



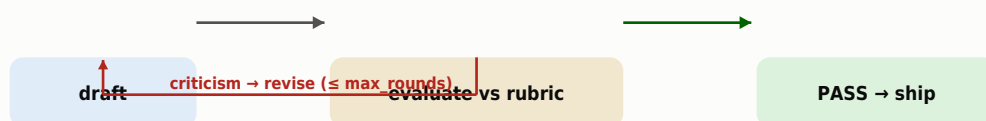
3 · PARALLELIZATION — slowest-scout wall clock (sectioning) · majority (voting)



4 · ORCHESTRATOR-WORKERS — the model writes the plan; code steers it



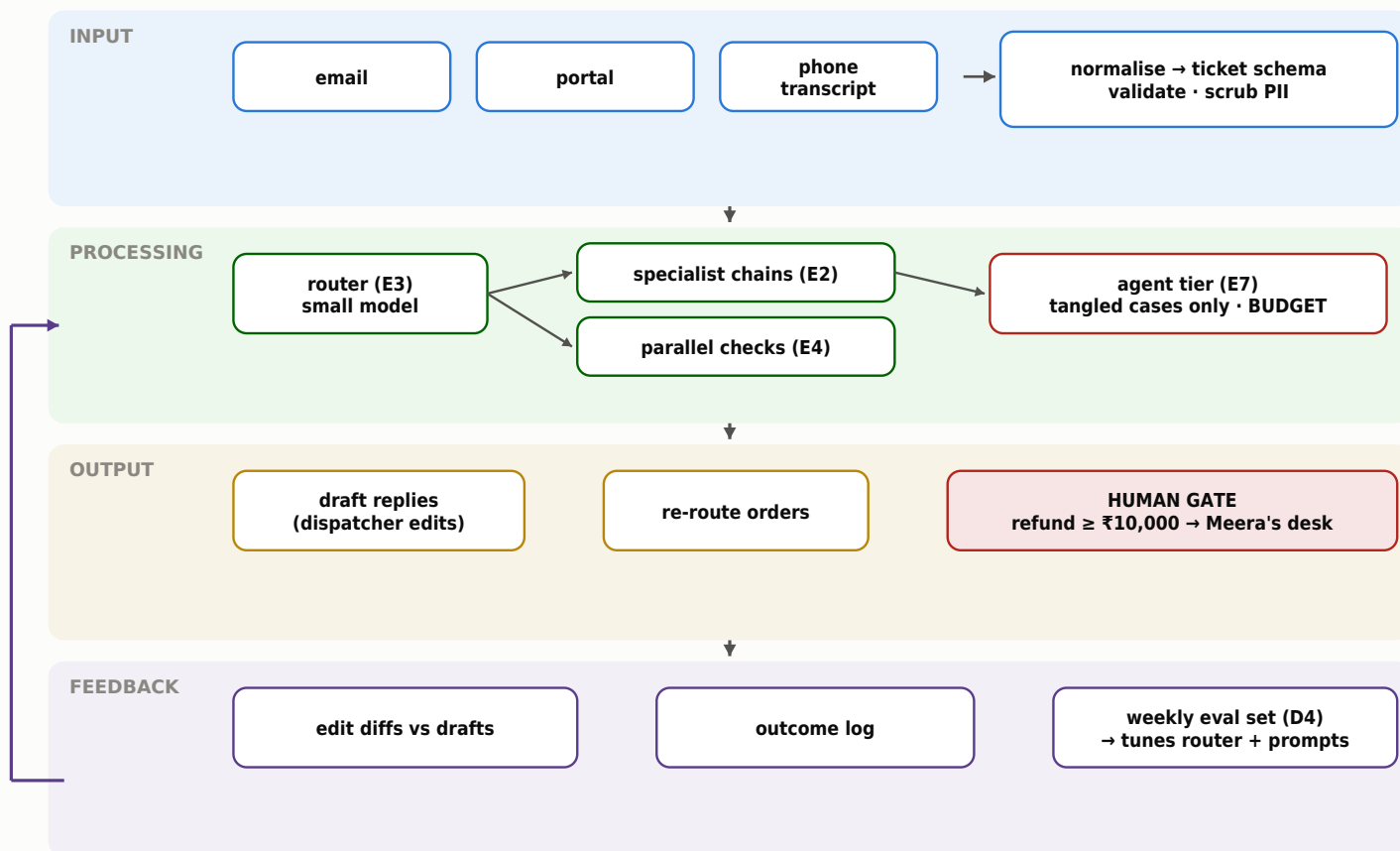
5 · EVALUATOR-OPTIMIZER — rubric loop, budget + PASS exit



ATLAS A3

The Kaveri Control Tower — the reference architecture

The end-to-end picture every 1.2 stem describes in words. Four stages; the human gate and the eval taps drawn in, because the exam's wrong options draw them out.



The purple arrow is the answer to a whole family of stems: without the feedback arc, launch quality is forever. When a 1.2 option list contains “add a feedback loop from dispatcher edits”, check whether the described architecture already has one — if not, that option beats every clever processing upgrade on the page.

ATLAS A4

The pattern selector — constraints in, pattern out

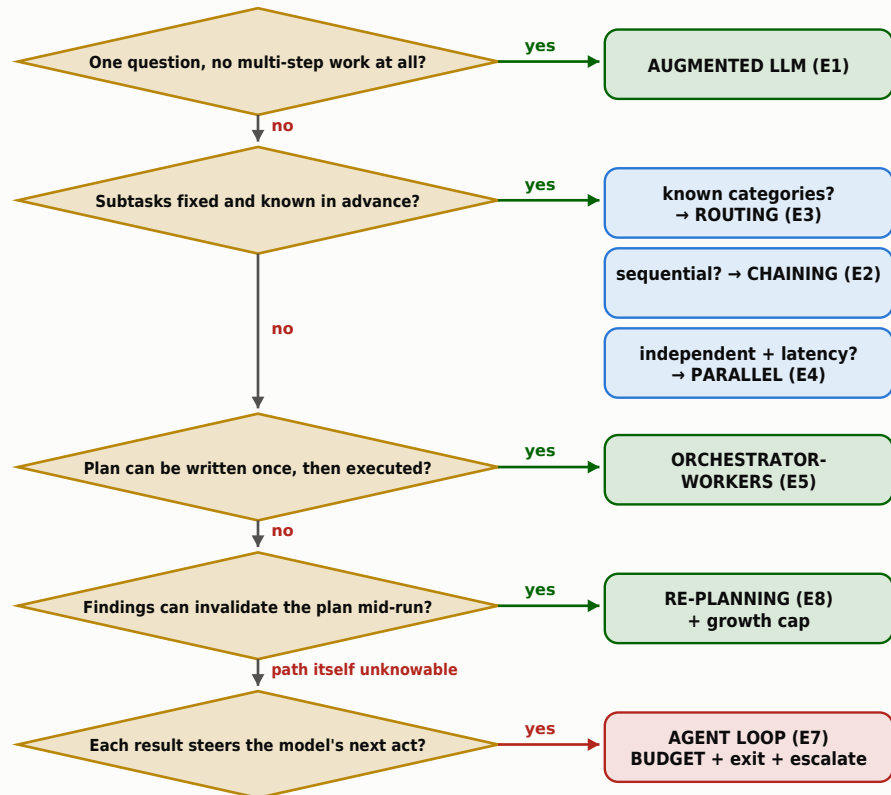
Day C's exercise: redraw this from memory. It is the 1.3 decision as a walk, not a memory feat.

Overlay questions,
asked at EVERY node:

rubric exists + drafts
improve? → add E6

high stakes + 3x cost OK?
→ add voting

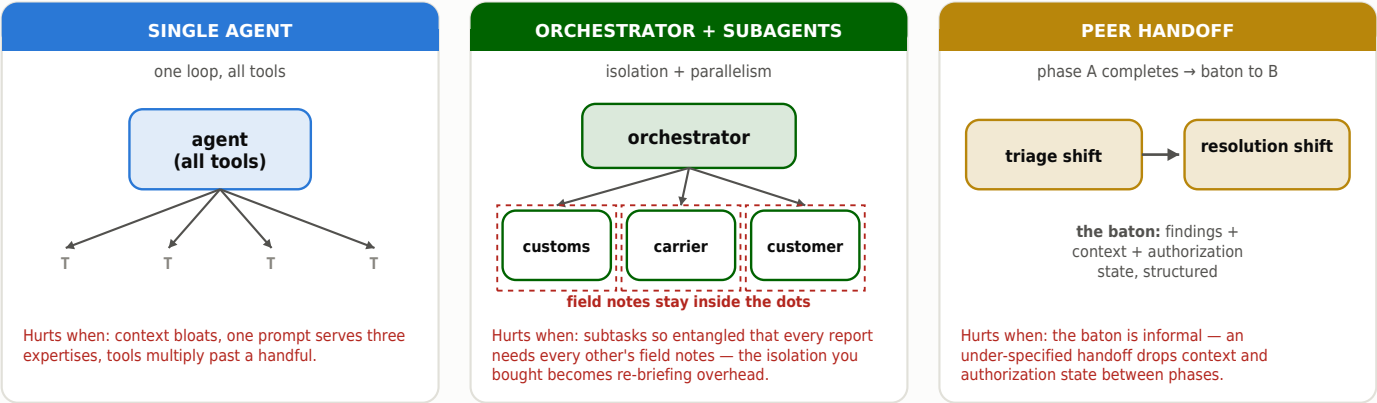
money / law / people?
→ human gate (D5)



ATLAS A5

Multi-agent topologies — and when each hurts

Three shapes for 1.4. The dots on the field-office walls are the point: notes stay inside; only reports travel.

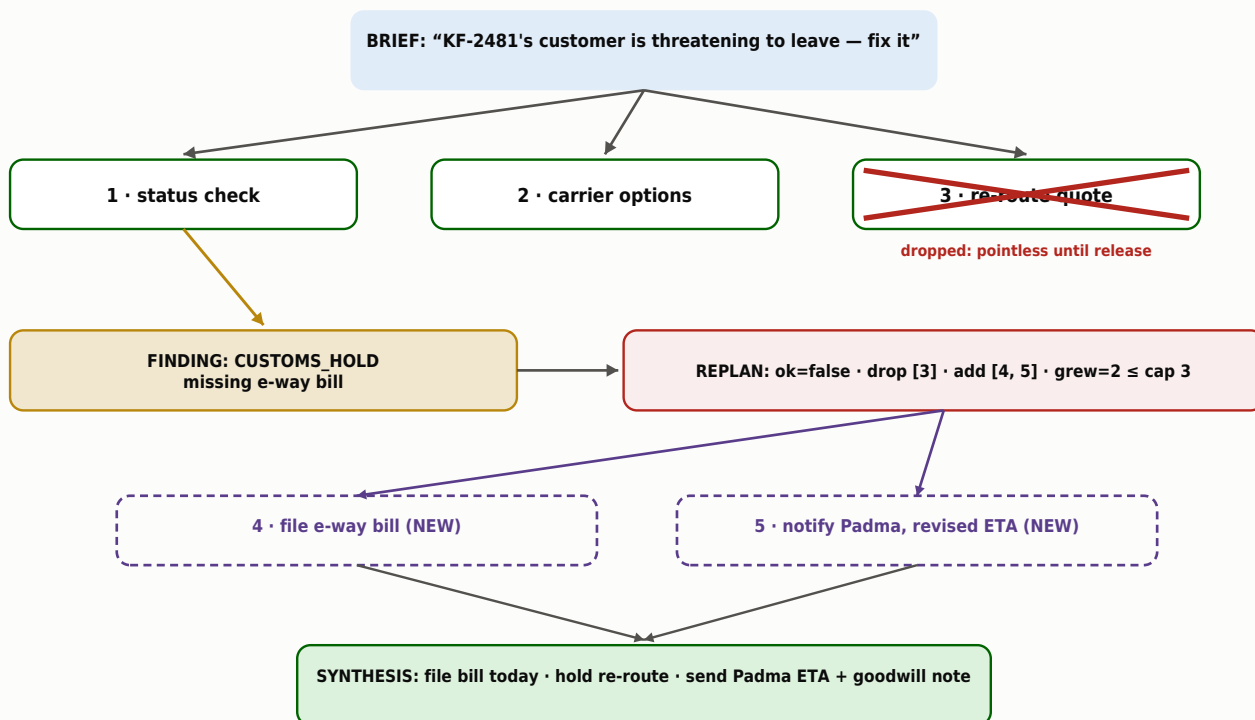


Caps to quote in answers (current SDK): spawn depth default 3 · concurrency default 20 · spend cap max_budget_usd refuses new spawns, stops background subagents, ends the query. Fleet-scale Law 3.

ATLAS A6

The decomposition tree — with the branch that grew

KF-2481's actual run from E8. Solid boxes were planned; the dashed ones appeared because a finding said so. The crossed one died the same moment.



© THE TELL

Static vs dynamic in one glance: if this tree could have been drawn before the run, chaining sufficed and orchestrator-workers was OE. The dashed boxes are the exam's evidence that dynamic machinery earned its keep.

ATLAS A7

The value-pillar matrix — three options, five pillars, every cell a number

Boardroom Tuesday: three candidate architectures for the control tower, scored the only way a 1.6 stem accepts — numerically, per pillar. Gold = meets Meera's target; crimson = breaches it.

Pillar · target	A · One big agent (every ticket → E7)	B · Tiered: route + chains, agent for Car	C · Rules 20% (no LLM, status quo)
Efficiency · first touch ≤ 5 min	~2 min ✓	~1 min ✓ (router is fast)	4 h ✗ unchanged
Cost · ≤ ₹6 / exception avg	~₹22 ✗ agent loops on trivia	~₹4.10 ✓ small models at door	~₹0 ✓ but see efficiency
Productivity · 3× dispatcher throughput	~2.8× ✓	~3.1× ✓	1× ✗
Performance / SLA · 99% drafts ≤ 30 s	~91% ✗ loop latency tail	~99.3% ✓ agent tier is the 1% tail	n/a — no drafts
Transformation · new service lines from patterns	logs exist, unmined	customs pre-clearance line from feedback log	none

Option B wins not because it is cleverest but because it is the only column with no crimson cell — **alignment means no pillar breached, not one pillar maximised**. Option A is the magnificent bridge: it beats B on nothing Meera measures and loses on two pillars. That asymmetry — capable but misaligned — is how 1.6 writes its best distractors.

⚠ TRAP · THE UNPRICED OPTION

When one option's cells cannot be filled with numbers at all (“improves experience”, “future-proof”), it is not a candidate — it is a wish. On the exam, an option you cannot score against the stated targets loses to any option you can, even a humbler one.

PART THREE

The Blueprint Book — 40 questions

Sat cold on Day B · one sitting · ~75 min · key sealed until Q40 · underline the tell before the options

PART THREE

The Blueprint Book — instructions, then forty questions

Six per objective; eight for 1.3 and 1.5. Twelve are head-to-head pairs where two options look right; eight carry a number that decides. Rules of the sitting: (1) underline the deciding clause in the stem before reading options; (2) mark, don't dwell — 110 seconds each; (3) the key stays sealed; (4) afterwards, every miss gets a ledger row with a family code.

- 1** Meera forwards a board note: “Deploy AI across operations this quarter.” Arjun’s first deliverable is due Friday. Which deliverable best serves objective 1.1? objective 1.1

 - A. A working prototype of an exception-triage agent, to make the ambition concrete
 - B. A problem statement naming the workload (1,200 exceptions/day), the constraint (first touch $\leq$ 5 min), and the measure of better
 - C. A comparison of Claude model tiers with per-token pricing for the board
 - D. A survey of which competitor logistics firms have deployed LLM systems
- 2** An exception category turns out to be fully deterministic: “if carrier = FastKart and status = RTO, re-book via BlueDart.” The team proposes adding it to the Claude router so all categories live in one place. What should Arjun say? objective 1.1

 - A. Agree — a single LLM pipeline is simpler to operate than a hybrid
 - B. Agree, but use the cheapest model for that category
 - C. Refuse — a deterministic rule belongs in code; an LLM adds cost and new failure modes to a solved problem
 - D. Refuse — this category should instead get a dedicated fine-tuned model
- 3** Kaveri's sortation belt needs a decision every 40 ms on package diverts, from structured scanner data. A vendor pitches “an agentic AI layer” for it. Why is this the wrong seat for an LLM? objective 1.1

 - A. The belt's data volume is too large for any API
 - B. Hard real-time control loops need deterministic sub-second latency on structured data — neither is an LLM property
 - C. LLMs cannot process scanner data formats
 - D. It is the right seat, provided a fast small model is chosen
- 4** Dispatchers currently resolve ambiguous “address” exceptions by phoning the customer. Meera wants them automated end-to-end. The historical log shows 38% of these end with the customer changing the address in ways the original ticket never contained. Best translation of this problem? objective 1.1

 - A. Full automation: the model drafts and sends address corrections
 - B. Claude drafts the outreach and proposed correction; the customer's reply closes the loop — automation of the draft, not the decision
 - C. Route address exceptions to a bigger model for higher accuracy
 - D. Exclude address exceptions from the system entirely
- 5** Which rewrite of Meera's ask is exam-grade for 1.1? objective 1.1

 - A. “Use Claude to make the exception desk intelligent”
 - B. “Reduce operational costs using agentic AI”
 - C. “Triage ~1,200 exceptions/day into 6 categories with a draft reply, first touch under 5 min, refunds above ₹10,000 always reaching a human”
 - D. “Deploy a multi-agent architecture with RAG over SOPs”
- 6** A pilot must prove value in 4 weeks on one exception type. The candidate types: refunds (high stakes, 9% of volume, heavy judgement) and delay notifications (low stakes, 55% of volume, template-like). For a first pilot whose goal is demonstrating measurable value fast, choose and justify. objective 1.1

 - A. Refunds — highest judgement content shows Claude at its best
 - B. Delay notifications — high volume and low stakes give fast, safe, measurable wins
 - C. Both, to maximise learning
 - D. Neither; run a 4-week requirements study first
- 7** Arjun's control-tower diagram shows: channels → normaliser → router → specialist chains → drafts to dispatchers. Meera asks “what's missing before I take this to the board?” The most defensible answer: objective 1.2

 - A. A bigger model for the router
 - B. A feedback arc — dispatcher edit diffs and outcomes flowing into a weekly eval set
 - C. A second router to verify the first
 - D. Caching to reduce cost per exception

- 8** Where in the control tower should PII scrubbing and input validation live? objective 1.2
- A. Inside each specialist prompt, as instructions to ignore PII
 - B. At the normaliser, before any model call
 - C. At output, before drafts reach dispatchers
 - D. In the agent tier only, where stakes are highest
-
- 9** The refund flow outputs auto-sent emails for amounts under ₹10,000 and pauses for Meera's desk above it. A stem option proposes removing the pause “since eval accuracy reached 97%.” Choose the architectural reading. objective 1.2
- A. Remove it — 97% clears any reasonable bar
 - B. Keep it — the gate is a designed output-stage control for irreversible, high-stakes actions; accuracy does not retire it
 - C. Replace the human with a second model as gate
 - D. Lower the threshold to ₹5,000 as a compromise
-
- 10** Three input channels (email, portal, phone transcripts) currently enter three separate pipelines with three prompts each. Defects cluster in phone transcripts. The 1.2-correct refactor: objective 1.2
- A. Improve the phone-transcript prompts
 - B. Normalise all channels into one ticket schema at input; run one pipeline; fix transcript quality at the normaliser
 - C. Add an agent to handle phone transcripts adaptively
 - D. Drop phone-channel automation
-
- 11** Which metric pair best instruments the feedback stage (not the processing stage)? objective 1.2
- A. Router accuracy and mean tokens per call
 - B. Dispatcher edit-distance on drafts, and re-open rate of closed exceptions
 - C. API latency p99 and cache hit rate
 - D. Model refusal rate and tool-call error rate
-
- 12** A reviewer notes the architecture has no path for an exception the system cannot classify at all. The 1.2-correct addition: objective 1.2
- A. Retry classification with temperature raised
 - B. A default route to a human queue, logged and counted, with the ticket intact
 - C. An 'other' category handled by the biggest model
 - D. Drop unclassifiable tickets after three retries
-
- 13** A stem describes: “The steps are identical for every damage claim: extract fields, check policy, draft response. Accuracy matters more than speed.” Choose the pattern. objective 1.3
- A. Agent with tools, so it can adapt per claim
 - B. Prompt chaining with programmatic gates between steps
 - C. Orchestrator-workers to decompose each claim
 - D. Single augmented-LLM call with all three instructions
-
- 14** Same claim flow, but now: “each claim's investigation depends on what the previous lookup reveals; some need two lookups, some need seven; the sequence cannot be scripted.” Choose. objective 1.3
- A. Prompt chaining with more gates
 - B. Routing with a 'complex' ward
 - C. An agent loop with a turn budget and human escalation
 - D. Parallel sectioning of all possible lookups
-
- 15** KAVI's loop reads: while True: r = create(...); if r.stop_reason == 'tool_use': run_tools() else: return r.content[0].text. Name the two defects. objective 1.3
- A. No budget on the loop; every non-tool_use stop_reason is treated as a finished answer
 - B. Missing cache configuration; wrong model for looping
 - C. Tools should be run in parallel; text should be JSON
 - D. while True is fine but the return should retry once
-
- 16** During a live run, a response returns stop_reason='max_tokens' mid-recommendation. The correct handling: objective 1.3
- A. Return the text — the model chose to stop
 - B. Append the assistant turn, send 'Continue.', and resume within the budget
 - C. Restart the investigation from turn one
 - D. Raise an exception; truncation is an API error

- 17** Which sequence of stop_reason values is the signature of a healthy two-tool investigation that ends naturally? objective 1.3
- A. tool_use → tool_use → end_turn
 - B. end_turn → tool_use → end_turn
 - C. tool_use → pause_turn → refusal
 - D. tool_use → tool_use → max_tokens
-
- 18** A team wants “an agent, because the demo impressed the board,” for choosing which of six SOP templates a routine notification should use. Volume: 30,000/day. Latency budget: 2 s. The 1.3 answer: objective 1.3
- A. Agent — board alignment matters
 - B. Routing via a small fast classifier; the agent demo is irrelevant to this workload
 - C. Evaluator-optimizer to polish each notification
 - D. Agent, but with a strict 2-turn budget to fit the latency
-
- 19** Two options survive elimination: (B) chaining with a validation gate; (C) single augmented-LLM call with tools. The stem's only remaining clause: “a malformed intermediate result must never reach the customer-facing step.” Pick and say why. objective 1.3
- A. B — only a programmatic gate between separate steps can guarantee interception
 - B. C — fewer calls mean fewer places to be malformed
 - C. Either; the difference is stylistic
 - D. C with a stronger system prompt forbidding malformed output
-
- 20** Which situation makes the plain augmented LLM (E1) the correct final answer, not a stepping stone? objective 1.3
- A. A one-shot question per ticket, tools at hand, no multi-step work, 50 ms overhead budget over the model call
 - B. A six-step review process with compliance sign-off
 - C. Unpredictable multi-tool investigations
 - D. A/B-tested draft improvement against a rubric
-
- 21** The customs subagent must read 40 SOP documents to answer one orchestration question. The orchestrator's context is already tight. Which multi-agent property is doing the work when this still scales? objective 1.4
- A. Parallelism
 - B. Context isolation — the subagent's reading stays in its own window; only the final answer returns
 - C. Tool restriction
 - D. Specialised prompts
-
- 22** A customer-facing subagent is instructed in its prompt: “never process refunds.” The 1.4-grade hardening: objective 1.4
- A. Repeat the instruction in capitals at the top of the prompt
 - B. Omit process_refund from the subagent's tool list, so the capability does not exist in its session
 - C. Add an evaluator model checking its outputs for refund attempts
 - D. Log every tool call for weekly review
-
- 23** An orchestrator delegates “check whether KF-2481 can be re-routed” to a fresh subagent, which returns: “Cannot answer — which consignment? which constraints?” The defect is in: objective 1.4
- A. The subagent's model tier
 - B. The delegation brief — a fresh subagent has no parent history, so the brief must carry ids, findings and constraints
 - C. The subagent's tool list
 - D. The topology; peer handoff should replace delegation
-
- 24** Three independent reviews (safety, cost, customer-tone) must run on each proposed resolution, currently 90 s sequentially at 30 s each. SLA: 40 s. Choose. objective 1.4
- A. A faster single reviewer model
 - B. Three specialist subagents run concurrently; slowest defines wall clock (~30 s)
 - C. Randomly sample one review per resolution
 - D. Raise the SLA
-
- 25** When does peer handoff beat orchestrator-subagents? objective 1.4
- A. Whenever subagents would exceed three
 - B. Sequential phases with a clean break — triage shift completing, then resolution shift owning the case — where no central synthesis is needed
 - C. Whenever latency matters
 - D. When subagents distrust each other
-

- 26** A stem proposes “a supervisor agent to manage the orchestrator, for governance.” Current SDK reality and the exam-correct reply: objective 1.4
- A. Accept; more oversight is safer
 - B. Reject the extra layer; enforce governance with the platform's own caps — spawn depth, concurrency limits, and a per-query spend budget
 - C. Accept, but make the supervisor a small model
 - D. Reject; subagents cannot be nested at all
-
- 27** The stem lists a task's four subtasks explicitly, in order. An option offers orchestrator-workers “so the model can decompose optimally.” Verdict: objective 1.5
- A. Take it — model decomposition is usually better
 - B. Reject — the decomposition already exists in the stem; paying a model to rediscover it is over-engineering
 - C. Take it for future-proofing
 - D. Reject — orchestrator-workers cannot handle four subtasks
-
- 28** Which brief property is the positive trigger for model-written decomposition (E5)? objective 1.5
- A. The brief is long
 - B. Subtasks cannot be predicted from the brief; they depend on what each case contains
 - C. The team lacks Python skills
 - D. Latency is unconstrained
-
- 29** In E8, the growth cap (`max_new=3`) exists because: objective 1.5
- A. The API limits added tasks to three
 - B. A plan that can grow needs a bound, or re-planning becomes an unbounded loop in planner's clothing
 - C. Three is the optimal subtask count
 - D. Workers cannot handle more than three new tasks
-
- 30** Mid-run, subtask 2's result reveals KF-2481 is in a customs hold; subtask 3 (re-route quote) is now pointless until release. Dynamic decomposition done right does what? objective 1.5
- A. Completes subtask 3 anyway — plans are commitments
 - B. Drops 3, adds 'file e-way bill' and 'notify customer of revised ETA', within the growth cap
 - C. Aborts and asks a human to write a new plan
 - D. Re-runs subtask 2 to confirm the hold
-
- 31** The replan call in E8 sends the model only {finding, remaining tasks} — not the full history. Why is this the right scope? objective 1.5
- A. It hides errors from the model
 - B. The replan question is local — does the remaining plan survive this finding? — so a focused, cheap, structured call answers it best
 - C. Full history would exceed context limits
 - D. The API forbids resending history
-
- 32** Choose the pair that correctly matches uncertainty level to machinery. objective 1.5
- A. Knowable subtasks → orchestrator-workers; unknowable → chaining
 - B. Knowable → chaining in code; plan-writable-once → orchestrator-workers; findings-can-invalidate-plan → re-planning with a cap
 - C. All levels → agent loop, tuned by budget
 - D. All levels → chaining, with more gates for uncertainty
-
- 33** A worker in E5 needs findings from subtasks it doesn't depend on, “for context.” Granting it would: objective 1.5
- A. Improve synthesis quality — approve
 - B. Break the decomposition's point — dependencies define what flows; everything-to-everyone recreates the monolith
 - C. Be required by the SDK
 - D. Halve cost by avoiding re-derivation
-
- 34** Head-to-head: (A) an agent loop (E7) or (B) orchestrator + re-planning (E8) for a nightly batch that must resolve 400 stuck consignments, each independent, results synthesised into one morning report for Meera. Deciding clause? objective 1.5
- A. A — agents handle stuck things
 - B. B — independent cases fan out to workers, re-planning absorbs surprises per case, and the fixed synthesise step produces the one report the brief demands
 - C. A — batches need a single loop
 - D. Either — they are equivalent at 400 scale

- 35** Which statement is exam-grade pillar alignment, not a wish? objective 1.6
- A. "The control tower will transform customer experience"
 - B. "First touch drops from 4 h to under 5 min at $\leq$ ₹6 per exception, measured weekly from the outcome log"
 - C. "Claude makes dispatchers more productive"
 - D. "The architecture is future-proof and scalable"
-
- 36** The stem fixes a ₹6/exception cost ceiling. One option: "a 3-model voting ensemble raising refund accuracy from 94% to 96.5%, at ₹19/exception on the 9% refund slice, blended ₹7.10 overall." Verdict: objective 1.6
- A. Accept — accuracy on refunds is worth it
 - B. Reject — the blended cost breaches the stated ceiling; the stem's pillar is the contract
 - C. Accept if Meera approves retroactively
 - D. Reject — voting never suits refunds
-
- 37** Kaveri's board asks which pillar the weekly customs-hold pattern report serves, and why it matters architecturally. objective 1.6
- A. Cost — reports are cheap
 - B. Transformation — mined feedback becomes a new pre-clearance service line, which is why the feedback log is a product, not exhaust
 - C. Efficiency — reports save reading time
 - D. SLA — reports prove compliance
-
- 38** Option A (one big agent) beats option B (tiered) on zero measured pillars and loses on cost (₹22 vs ₹4.10) and SLA (91% vs 99.3%). A stakeholder prefers A as "more ambitious." The 1.6 response: objective 1.6
- A. Adopt A; ambition drives adoption
 - B. Adopt B; alignment means no pillar breached — capability that no measured pillar rewards is cost, not value
 - C. Compromise: run A for refunds, B elsewhere
 - D. Re-run the measurements until A wins one
-
- 39** A proposal claims productivity gains but offers no baseline measurement of current dispatcher throughput. The 1.6-correct first step: objective 1.6
- A. Deploy, then measure the improvement
 - B. Measure the baseline now — without it, no post-deployment number can attribute the gain
 - C. Use industry-standard throughput figures
 - D. Skip productivity; use the cost pillar instead
-
- 40** Meera gets 90 seconds with the board. Which architecture summary earns the next meeting? objective 1.6
- A. "We built a multi-agent RAG system with MCP integration and subagent isolation"
 - B. "Exceptions answered in 5 minutes instead of 4 hours, at ₹4 each, with every refund above ₹10,000 still crossing a human desk — and the logs are already showing us a customs product"
 - C. "We followed Anthropic's canonical workflow patterns"
 - D. "The system uses the latest models with strict guardrails"

SEALED

THE KEY

Do not turn this page until all forty answers are marked.

Scoring: count correct per objective, not just the total. 30+/40 with no objective below 4/6 (or 5/8) = on track.

THE KEY

Answers, tells, and why each distractor fails

Every explanation names the tell (underlined) and tags the wrong options with a miss family: **OE** Over-engineered · **VG** Vocabulary gap · **TR** Trade-off reversed · **CM** Constraint missed · **HL** Human loop skipped · **SF** Silent failure · **SD** Sounds-diligent.

- Q1** **B** **The tell:** “Deploy AI” is ambition, not a problem — the note names no workload, constraint, or measure. **obj 1.1**
Why B: 1.1 is translation: workload + constraint + measure of better, before any building.
A [OE] building before the problem is stated; the prototype's shape is a guess
C [CM] model choice is objective 2.1, and premature before the problem exists
D [VG/SD] sounds diligent, decides nothing about Kaveri's own workload
- Q2** **C** **The tell:** The rule is already fully deterministic — there is no judgement left for a model to add. **obj 1.1**
Why C: Claude-shaped work is judgement over unstructured input. Deterministic logic stays in code.
A [OE] uniformity is not a reason to pay per token for an if-statement
B [OE] cheaper waste is still waste, and adds a failure mode
D [OE] fine-tuning is the most expensive way to reimplement an if-statement
- Q3** **B** **The tell:** 40 ms, hard control loop, structured data — three disqualifiers in one clause. **obj 1.1**
Why B: Objective 1.1 includes recognising non-fits. Real-time structured control is one.
A [CM] volume isn't the disqualifier; latency and determinism are
C [VG] format is trivially handled; the issue is timing guarantees
D [TR] no model tier makes latency deterministic at 40 ms
- Q4** **B** **The tell:** 38% require information that is not in the ticket — no model call can conjure it. **obj 1.1**
Why B: Translate to what the data supports: draft + human/customer completion where information is missing.
A [HL] sends corrections invented without the missing information
C [TR] accuracy isn't the gap; missing information is
D [CM] over-retreat; the draft-and-ask step automates most of the toil
- Q5** **C** **The tell:** Only one option names workload, categories, numbers, and a human gate. **obj 1.1**
Why C: Workload + constraint + measure + gate: the four marks of a translated problem.
A [VG] “intelligent” is ambition-speak
B [SD/VG] a pillar with no number and no workload
D [OE] an architecture is an answer; 1.1 wants the question stated
- Q6** **B** **The tell:** The goal is measurable value in 4 weeks — volume × safety beats drama. **obj 1.1**
Why B: Pilot selection is 1.1 applied: the constraint (4 weeks, measurable) picks the workload.
A [TR] optimises impressiveness where the stem fixed speed-to-value and risk
C [OE] two pilots in 4 weeks halves both
D [SD] a study produces no measured value by week 4
- Q7** **B** **The tell:** The described flow ends at output — nothing returns to improve the system. **obj 1.2**
Why B: 1.2 = input → processing → output → feedback. Wrong options upgrade present stages; right ones add absent arcs.
A [TR] capability upgrade where the gap is a missing stage
C [OE] verification circle; adds cost, closes no loop
D [CM] caching is real but optimises a stage that exists; the absent stage outranks it
- Q8** **B** **The tell:** Scrub before any model call — after is too late by definition. **obj 1.2**
Why B: Config beats instruction, and placement beats both: validation is an input-stage property.
A [HL/SD] instructions are persuasion; PII already left the building
C [CM] output scrubbing means every model already saw the PII
D [CM] the routine 80% would ship PII unscrubbed
- Q9** **B** **The tell:** Irreversible + money defines the gate's purpose; 97% still mis-approves 3 in 100. **obj 1.2**
Why B: Human gates are architecture, not tuning parameters. They move only with the risk, not the accuracy.
A [HL] the classic; accuracy is an argument, the gate is a control
C [HL] a model gating a model is not a human in the loop
D [TR] moves the number the wrong way for no stated reason

Q10	B	The tell: <u>Three pipelines × three prompts</u> is the smell; defects clustering at one channel points at input, not processing. Why B: End-to-end design pushes variability to the edges: one schema in, one pipeline through. A [CM] patches one pipeline, keeps the 3× maintenance surface C [OE] adaptivity where the problem is inconsistent input shape D [CM] retreat where a normaliser solves it	obj 1.2
Q11	B	The tell: Feedback measures <u>what happened after output met the world</u>. Why B: Edit diffs and re-opens are the world grading the system — the raw material of next week's eval set. A [CM] processing metrics, valuable but the wrong stage C [CM] infrastructure metrics D [CM] model-behaviour metrics, still upstream of the world	obj 1.2
Q12	B	The tell: <u>Cannot classify at all</u> asks for an exit, not a better attempt. Why B: Every flow needs its escalation floor — the E7 lesson at architecture scale. A [SF] retries defer the same failure C [OE] a bigger guess is still a guess with no floor under it D [SF] dropped tickets are silent failures with customers attached	obj 1.2
Q13	B	The tell: <u>Identical steps every time; accuracy over speed</u> — the chaining signature verbatim. Why B: Known fixed path + accuracy-over-latency = chaining. Recite it. A [OE] adaptivity is the property the stem just excluded C [OE] the decomposition is already given, in the stem D [TR] one call trades away the gates that buy the accuracy the stem prioritises	obj 1.3
Q14	C	The tell: <u>The sequence cannot be scripted</u>; each result steers the next act. Why C: This pair with the previous question is the 1.3 hinge: the constraint flipped, so the pattern flips. The agent IS correct here. A [CM] a fixed chain cannot express an unscriptable sequence B [CM] routing picks a ward; it does not conduct an investigation D [OE/TR] running all possible lookups buys latency and cost to avoid the loop the problem is	obj 1.3
Q15	A	The tell: <u>while True</u> and <u>else-that-swallows</u> — the two organs both missing. Why A: Budget + exit + total handling of stop_reason. max_tokens as 'done' ships half a verdict. B [CM] real topics, wrong objective; the loop's control flow is the defect C [CM] optimisations on top of a broken loop D [SF] a blind retry treats refusal and truncation identically, again	obj 1.3
Q16	B	The tell: <u>max_tokens</u> means the page ran out, not the thought. Why B: Truncated ≠ finished. File what arrived, ask for the rest, stay inside the budget. A [SF] ships a truncation as a verdict C [TR] pays the whole investigation again for one more page D [VG] it arrives as HTTP 200; a stop_reason is a status, not an error	obj 1.3
Q17	A	The tell: Fuel, fuel, <u>natural exit</u>. Why A: Read traces like ECGs: tool_use is the beat, end_turn the rest state. B [VG] end_turn ends the loop; nothing follows it in one investigation C [CM] pause/refusal are handleable but neither is 'healthy natural end' D [SF] ending on max_tokens means the verdict was cut off, not delivered	obj 1.3
Q18	B	The tell: <u>Six known templates. 30,000/day. 2 s</u> — three constraints, all shouting router. Why B: Law 1 in exam form. The constraint picks; the demo does not vote. A [OE] ambition wearing a lanyard C [OE] no rubric problem was stated; latency budget forbids loops D [OE] a capped agent is still loop latency and loop cost for a lookup-shaped task	obj 1.3
Q19	A	The tell: <u>Must never reach</u> = a guarantee — only code between steps gives one. Why A: Head-to-head resolved by one clause. Guarantees live in code; the pattern with the gate wins. B [TR] fewer calls, but the malformed result now travels inside one call, uninspectable C [CM] the guarantee clause is precisely the non-stylistic difference D [SD] a prompt is persuasion; 'never' needs a control	obj 1.3

Q20	A	The tell: <u>No multi-step anything</u> — the cell suffices; arranging cells would add only overhead. Why A: Simplest-possible is a real answer, not a warm-up. Expect it once per paper. B [CM] six steps and sign-off is chaining with a human gate C [CM] that's the agent's home ground D [CM] that's evaluator-optimizer with D4 machinery	obj 1.3
Q21	B	The tell: <u>40 documents read, context tight</u> — the reading must happen somewhere that isn't the parent. Why B: Isolation is token economics solved architecturally: field notes stay in the field office. A [CM] nothing here runs concurrently C [CM] permissions aren't the pressure; tokens are D [CM] expertise helps answer, not fit	obj 1.4
Q22	B	The tell: "Never" by <u>instruction</u> is persuasion; by <u>tool-list omission</u> it is physics. Why B: Config beats instruction — the law of laws, wearing 1.4 clothes. A [SD] louder persuasion C [OE] a watcher over a door that could simply not exist D [SD] detects Tuesday's breach on Friday	obj 1.4
Q23	B	The tell: <u>Fresh subagent</u> = empty inheritance; the question's vagueness is the orchestrator's fault. Why B: Structured handoffs carrying context + authorization state: your 100% row is the fix for this stem. A [TR] capability can't recover information never sent C [CM] tools were never the missing piece; context was D [OE] changing shape to dodge writing a proper brief	obj 1.4
Q24	B	The tell: <u>Independent</u> + <u>40 s against 3 x 30 s</u> — the parallel signature with numbers. Why B: E4's arithmetic at agent scale: concurrency turns a sum into a max. A [TR] one model doing three expertises faster is a hope, not a design C [CM] the stem says all three must run D [CM] the SLA is the contract, not the variable	obj 1.4
Q25	B	The tell: <u>Clean sequential break, no synthesis</u> — a baton, not a hub. Why B: Topology follows the work's shape: hub when synthesis, baton when succession. A [VG] no such numeric rule exists C [CM] latency argues for parallel, which is the hub shape D [VG] trust is not a topology criterion	obj 1.4
Q26	B	The tell: <u>A coordinator over the coordinator</u> — governance is what caps are for. Why B: Depth 3, concurrency 20, max_budget_usd: quote the caps, delete the committee. A [OE] the S3 ghost, promoted C [OE] a cheaper unnecessary layer is still unnecessary D [VG] nesting exists (default depth 3); the point is capping it, not denying it	obj 1.4
Q27	B	The tell: <u>Subtasks listed in the stem itself</u> — the plan pre-exists. Why B: Law 4, negative form: never dynamically decompose the known. A [OE] rediscovery as a service C [OE] futures are not constraints D [VG] it handles four fine; it's just unnecessary	obj 1.5
Q28	B	The tell: <u>Cannot be predicted</u> — the one phrase that unlocks the foreman. Why B: The foreman writes the list only when nobody else can. A [VG] length is not uncertainty C [SD] org gaps don't select architectures D [CM] slack latency permits many things; it selects nothing	obj 1.5
Q29	B	The tell: Re-planning is a loop; <u>every loop needs a budget</u>. Why B: Law 3 applies to plans too. grew < max_new is the planner's BUDGET. A [VG] no such API limit C [VG] the number is a budget, not an optimum D [VG] workers are indifferent	obj 1.5

Q30	B	The tell: A finding invalidated part of the plan — the plan must answer the finding. Why B: This is your 75% row as a stem. Adapt: drop, add, count against the cap, continue. A [CM] executing a fixed sequence regardless of findings is the exact 75%-row error C [HL-inverse/TR] escalation is for exhausted budgets, not first detours D [SF-adjacent] re-verifying instead of acting defers the adaptation	obj 1.5
Q31	B	The tell: The question is <u>does the remaining plan still hold</u> — nothing else is load-bearing. Why B: Decomposition applies to prompts too: ask the smallest question that decides. A [VG] nothing is hidden; findings are the input C [CM] it might fit; scope is chosen for focus and cost, not fit D [VG] no such rule	obj 1.5
Q32	B	The tell: Escalating <u>uncertainty</u>, escalating machinery — never more than demanded. Why B: The three-tier ladder is the whole objective in one row. Memorise it as a ladder, not a list. A [Reversed] VG C [OE] one hammer, three differently priced nails D [CM] gates validate outputs; they cannot write absent plans	obj 1.5
Q33	B	The tell: <u>needs: []</u> is a design statement — data flows along declared dependencies only. Why B: Decomposition is as much about what does NOT flow as what does. A [SD] 'context' without a dependency is noise with prestige C [VG] no such requirement D [TR] tokens rise, focus falls; the opposite of the claim	obj 1.5
Q34	B	The tell: <u>400 independent + one synthesised report</u> — fan-out shape with a mandated closing step. Why B: When both candidates 'could work', the stem's output clause (one report) picks the pattern with a built-in synthesise. A [VG] vibes; one loop serialises 400 investigations C [CM] batch argues fan-out, not one loop D [CM] the synthesis mandate and the independence are exactly what makes them non-equivalent	obj 1.5
Q35	B	The tell: Pillar + <u>number</u> + measurement <u>mechanism</u>. Why B: Law 5: value is a pillar with a number attached — and a way to read the number. A [VG] pillar with no number C [VG] pillar with no number D [VG] two wishes in a trench coat	obj 1.6
Q36	B	The tell: <u>₹7.10 against a ₹6 ceiling</u> — one number decides. Why B: The stated pillar is the contract. Nobler physics than ordered earns zero marks. A [TR] the pillar swap, textbook form C [SD] the exam grades the stated constraint, not imagined renegotiation D [OE-inverse/VG] voting suits refunds fine; the ceiling is the problem	obj 1.6
Q37	B	The tell: <u>New service line from patterns</u> — the transformation pillar's definition. Why B: Transformation stems reward reading exhaust as ore. A [CM] prices the artefact, misses the pillar C [CM] wrong pillar, minor effect D [CM] compliance is evidence, not transformation	obj 1.6
Q38	B	The tell: A <u>beats B on nothing Meera measures</u> — the matrix already answered. Why B: The A7 matrix as a stem. No-crimson-column beats tallest-single-bar. A [SD] ambition is not a pillar C [OE] imports A's worst economics into the money category D [SD] measurement shopping	obj 1.6
Q39	B	The tell: A gain is a <u>difference</u>; a difference needs two numbers, and one must precede deployment. Why B: Pillar discipline starts before the build: baseline, target, mechanism. A [SF] the counterfactual dies at deployment C [SD] someone else's desk is not Meera's baseline D [TR] swapping pillars to dodge measurement	obj 1.6

Q40 **B****The tell:** Board time buys pillars and numbers, never mechanism names.

obj 1.6

Why B: 1.6's last word: translate architecture back into the boardroom's five words. That full circle is Domain 6's opening door.**A [VG]** technology names carry no pillar**C [SD]** provenance is not value**D [VG]** 'latest' and 'strict' are unpriced adjectives

Twenty-five things worth carrying into the room

THE TREE & THE LAWS

- 1 Taxonomy: **augmented LLM** → 5 workflows → agents. Workflows = predefined code paths; agents = model directs its own process.
- 2 Law 1: the **constraint** picks the pattern, not the ambition.
- 3 Law 2: workflows when the path is known; agents when it isn't.
- 4 Law 3: every loop needs a **budget + exit** (E6: max_rounds + PASS · E7: BUDGET + end_turn · E8: max_new).
- 5 Law 4: decompose on **discovery**, not in advance — code-decompose the known, model-decompose the unknowable, re-plan when findings invalidate.
- 6 Law 5: value = pillar + **number** + mechanism. Alignment = **no pillar breached**, not one maximised.

THE FIVE WORKFLOWS, ONE LINE EACH

- 7 **Chaining**: fixed steps + programmatic gates; accuracy over latency.
- 8 **Routing**: cheap classifier at the door, specialist ward (and model tier) per category.
- 9 **Parallel**: sectioning = coverage, wall clock = slowest scout; voting = confidence at 3× cost — judgement only, never facts.
- 10 **Orchestrator-workers**: the model writes the subtask list; code steers delegate → work → synthesise.
- 11 **Evaluator-optimizer**: needs a real rubric; criticism + draft go back together.

THE LOOP — YOUR 50% ROW

- 12 Seven stop_reason values: **end_turn** · **max_tokens** · **stop_sequence** · **tool_use** · **pause_turn** · **refusal** · **model_context_window_exceeded**. All HTTP 200 — status, never error.
- 13 tool_use = fuel: append assistant turn → run tools → tool_result by tool_use_id. **The model's reply decides whether there is a next turn.**
- 14 max_tokens = truncated, NOT done: file it, say "Continue." · refusal → different path, no blind retry · wildcard → escalate.
- 15 The floor: budget spent, case open → **human hand-off with a reason**. Never a silent stop.
- 16 Healthy trace: tool_use → tool_use → end_turn. The trap: an else that swallows.

DECOMPOSITION — YOUR 75% ROW

- 17 The ladder: subtasks knowable → **chain in code** · plan writable once → **E5** · findings can invalidate → **E8 re-plan + growth cap**.
- 18 Subtasks listed in the stem itself → never orchestrator-workers (OE).
- 19 Replan call sees only {finding, remaining} — the smallest question that decides.
- 20 Data flows along declared needs only; "for context" without a dependency is noise.

MULTI-AGENT & THE BOARDROOM

- 21 Subagent starts **fresh** — the brief must carry ids, findings, authorization state. Isolation: field notes stay in; only the report returns.
- 22 "Never do X" = omit the tool, not a louder prompt. Config beats instruction.
- 23 Caps, not committees: depth **3** · concurrency **20** · max_budget_usd. Coordinator-over-coordinator = OE; a subagent itself is often RIGHT.
- 24 Pillars: efficiency · cost · productivity · SLA · transformation. The stated pillar is the contract (₹7.10 > ₹6 loses, whatever it buys).
- 25 Board sentence shape: "5 min not 4 h, ₹4 each, refunds > ₹10k still cross a human desk."

indigo tokens · **gold** numbers · **crimson** traps · **green** correct moves · underline = the tell — read this page twice on Sun 1 Nov, and not the night before that.

Families: **VG** vocabulary gap · **OE** over-engineered · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped. Write the tell you skipped, in your own words — the ledger's value is the sentence, not the tick.

[illegible]

Pattern rule: three misses sharing a family beats three misses sharing an objective — fix the habit, not the topic. Three **CM** rows = slow down and underline before the options. Three **OE** rows = re-walk Atlas A4 before the next sitting.